

Outfit Lab

Dress one character together.

Every save, merge, send and undo runs real G



Outfit Lab

Open: _____

Open the link and type your name.
The app picks your lab.



QR

STEP 1**Save every version**

git commit

WHAT IT IS

a saved snapshot of the whole project.

WHAT IT DOES

records the exact state, who saved it, when, and the commit before it.

HOW GIT DOES IT

each file is stored as a blob, the folder as a tree. A commit object = tree ID + parent ID(s) + author + committer + message. Its ID is the SHA-1 of that object, so any change gives a new ID. Nothing is edited in place.

Git stores the name and clock your laptop gives it. It checks neither.

STEP 2**Try two ideas at once**

git branch

WHAT IT IS

a movable label on one commit.

WHAT IT DOES

lets you try an idea without touching main.

HOW GIT DOES IT

a branch is a tiny file (`.git/refs/heads/<name>`) holding one commit ID. Committing moves it forward. HEAD records which branch you are on. Creating a branch copies nothing.

In real Git, each of you would have your own clone with one working directory. Here your lab shares one repo.

STEP 3**Make one outfit from both**

git merge

WHAT IT IS

combining two lines of work.

WHAT IT DOES

keeps a change made on one side. It stops for a person when both sides changed the same or neighbouring lines.

HOW GIT DOES IT

Git finds the merge base (the last common commit) and compares each side with it (a 3-way merge), then writes a merge commit with two parents. If your branch has nothing new (the other side already contains it), Git fast-forwards: only the label moves, and no merge commit is made.

STEP 4**Put your outfit on the Wall**

git push / git pull

WHAT IT IS

sharing commits with a copy elsewhere (the Wall).

WHAT IT DOES

push sends your new commits and moves the remote branch.
pull brings theirs in.

HOW GIT DOES IT

copies exchange only the objects the other side lacks; IDs come from content, so copies agree without coordination. A push is accepted only if it is a fast-forward of the remote branch. Otherwise fetch first, then merge or rebase.

STEP 4**Put your outfit on the Wall**

git rebase

WHAT IT IS

replaying your commits on top of another branch.

WHAT IT DOES

gives a straight history with no merge commit.

HOW GIT DOES IT

for each of your commits, Git applies its change to the new base and writes a new commit. The new parent gives it a new ID. Author and author date are kept. The originals become unreachable and stay in your reflog for a while.

STEP 5**Oops: undo a shared mistake**

git revert / git reset

WHAT IT IS

two kinds of undo.

WHAT IT DOES

revert adds a commit that undoes an old one, and is safe on shared history. reset moves your branch label back, and is only safe if nobody else has those commits.

HOW GIT DOES IT

revert applies the opposite of the commit's change and commits it on top, so history only grows. reset rewrites the branch file. The commits left behind are still listed in the reflog (a local log of where each label pointed) until they expire and gc removes them.

STEP 6**The boss wants it clean**

Squash + `git push --force`

WHAT IT IS

cleaning up history.

WHAT IT DOES

squash turns several commits into one. Force push makes the Wall accept it anyway.

HOW GIT DOES IT

squash writes one new commit whose tree is the final state. The originals, with their authors and times, become unreachable. `--force` skips the fast-forward check and moves the remote branch. `git gc` later deletes unreachable objects.

Switching to Git: the Good, the Bad, and the Ugly

GOOD

Cheap branches.

Local commits and reverts.

Developers prefer a flat history.

BAD

Fast-forward forgets which branch a change came from.

Rebase rewrites the commit; the patch can change too.

Squash can drop the cards — and, once the branch is deleted, even who made them.

UGLY

Microsoft traces each change's route to main: its integration path.

For Git, that tracing had to be redesigned from scratch.

Some loss cannot be recovered.

For analysts, flat history is data loss.

Flat history helps developers find and revert a bad change (bisect, revert). It loses where a change came from and who made it.

THE PAPER

What you lived

Step 3: after the fast-forward, you deleted fancy. No card says which cards were made on it.

Step 4: Replay on top made a copy with a new ID; the original card is not on the Wall.

Step 6: the squash dropped the cards, and who made them.

Step 6: the 🥾 audit failed. The Wall no longer knows who added the boots.

PAIRS, IN THE APP

Your team squash-merges every feature branch and deletes it. What do you gain, and what can an auditor no longer answer?

Cards never change.
Sticky notes move.
The Wall copies cards.